

PROJETO ECOCOLETA

GABRIEL BRITO FALCÃO
HEMILY BARBUDA DE JESUS RAMOS
SOPHIA EDUARDA LIMA

RELATÓRIO DE EXECUÇÃO DE TESTES E COBERTURA DE CÓDIGO

Testes Unitários: Camada de Aplicação (Use Cases)

Data de execução: 01 de junho de 2026 | Ferramenta de build: Gradle 8.4 | Cobertura: JaCoCo 0.8.11

INTRODUÇÃO

Este relatório documenta a execução dos testes unitários do sistema EcoColeta, uma plataforma de agendamento de coleta seletiva desenvolvida com Spring Boot 3.2.5, Java 21 e arquitetura limpa (Clean Architecture). O objetivo é apresentar os resultados da execução dos testes, o comportamento verificado em cada caso de teste e as métricas de cobertura de código geradas pela ferramenta JaCoCo.

Os testes foram desenvolvidos com JUnit 5 e Mockito 5.x, focando exclusivamente na camada de aplicação (use cases), responsável por toda a lógica de negócio do sistema. A execução foi realizada em contêiner Docker isolado (gradle:8.4-jdk21-alpine) para garantir reprodutibilidade.

AMBIENTE DE EXECUÇÃO

Componente	Versão / Configuração
Linguagem	Java 21, OpenJDK 64-Bit Server VM
Framework de testes	JUnit 5 (JUnit Platform 1.10)
Framework de mocks	Mockito 5.x, via @ExtendWith(MockitoExtension.class)
Ferramenta de cobertura	JaCoCo 0.8.11
Build tool	Gradle 8.4
Contêiner de execução	Docker, imagem gradle:8.4-jdk21-alpine
Tipo de testes	Unitários, sem dependência de banco de dados, rede ou contexto Spring
Data e hora	01/06/2026 às 21:11:35 (horário de Brasília)

SUMÁRIO DOS RESULTADOS

Métrica	Valor
Total de testes executados	5
Testes aprovados (PASSED)	5
Testes com falha (FAILED)	0
Testes ignorados (SKIPPED)	0
Taxa de sucesso	100%
Tempo total de execução dos testes	35,5 segundos
Classes de teste executadas	3

Todos os cinco casos de teste foram executados com sucesso, sem nenhuma falha ou erro registrado. O tempo elevado (35,5 s) deve-se ao carregamento do agente byte-buddy na primeira suíte (AgendarColetaUseCaseTest); as demais suítes executaram em menos de 50 milissegundos.

RESULTADO DETALHADO POR CLASSE DE TESTE

AgendarColetaUseCaseTest

deveCriarAgendamentoEPublicarStatusAgendado()	PASSOU	0,263
deveLancarExcecaoQuandoSolicitacaoNaoExiste()	PASSOU	35,242

AtualizarStatusUseCaseTest

deveAtualizarStatusEPublicarEvento()	PASSOU	0,017
deveLancarExcecaoQuandoSolicitacaoNaoEncontrada()	PASSOU	0,019

CriarSolicitacaoUseCaseTest

deveCriarSolicitacaoComStatusPendente()	PASSOU	0,007
---	--------	-------

COMPORTAMENTOS VERIFICADOS

AgendarColetaUseCaseTest	deveCriarAgendamentoEPublicarStatusAgendado	Ao agendar uma coleta, o status da solicitação deve ser alterado para AGENDADO e o evento correspondente deve ser publicado no RabbitMQ. Verifica-se que o repositório salva o agendamento e o publisher é invocado.
	deveLancarExcecaoQuandoSolicitacaoNaoExiste	Quando a solicitação informada não existe no repositório, o use case deve lançar <code>IllegalArgumentException</code> . Nenhum agendamento deve ser persistido e o publisher do RabbitMQ não deve ser chamado.
AtualizarStatusUseCaseTest	deveAtualizarStatusEPublicarEvento	A atualização de status deve persistir o novo valor na solicitação e publicar um <code>StatusAtualizadoEvent</code> com o ID correto e o novo status, confirmando a integração entre o caso de uso e o publicador de eventos.
	deveLancarExcecaoQuandoSolicitacaoNaoEncontrada	Quando a solicitação a ser atualizada não existe, deve ser lançada uma exceção. O repositório não deve salvar nada e o publisher não deve ser invocado.
CriarSolicitacaoUseCaseTest	deveCriarSolicitacaoComStatusPendente	Toda solicitação de coleta criada deve ter o status inicial PENDENTE e o e-mail do solicitante deve ser apresentado corretamente. Verifica-se que o repositório é chamado com os dados corretos.

COBERTURA DE CÓDIGO POR JACOCO

A cobertura de código foi medida pela ferramenta JaCoCo 0.8.11 com o relatório gerado em formato XML e HTML. A estratégia adotada concentra os testes unitários na camada de aplicação (use cases), que encapsula toda a lógica de negócio. As demais camadas, como controllers REST, configuração de segurança, mensageria e persistência, dependem de infraestrutura externa e são adequadamente exercidas por testes de integração ou pela validação via interface Swagger.

Cobertura por Pacote

Pacote/Classe	Instruções	Total	Cobertura	Observação
application.agendamento: AgendarColetaUseCase	64	64	100%	16 linhas, 3 métodos; cobertura total
application.coleta: AtualizarStatusUseCase	42	42	100%	10 linhas, 3 métodos; cobertura total
application.solicitacao: CriarSolicitacaoUseCase	37	37	100%	12 linhas, 2 métodos; cobertura total
domain: Entidades e eventos de domínio	109	199	55%	Getters/setters exercitados indiretamente pelos testes de use case
adapters: Controllers REST	0	253	0%	Testado via Swagger e testes de integração
config: Segurança, JWT, CORS	0	231	0%	Requer contexto Spring; testado por integração
infrastructure.messaging: RabbitMQ	0	51	0%	Mockado nos testes unitários; broker real testado em integração
infrastructure.persistence: Repositórios JPA	N/A	N/A	N/A	Interfaces JPA sem implementação própria mensurável

Totais Gerais (Projeto Completo)

Instruções (bytecode)	261	652	913	28,6%
Linhas de código-fonte	67	146	213	31,5%
Complexidade ciclomática	26	76	102	25,5%
Métodos	26	71	97	26,8%
Classes	7	19	26	26,9%
Branches (desvios condicionais)	0	10	10	0%

Nota sobre os percentuais gerais: os valores globais de 28,6% (instruções) e 31,5% (linhas) refletem o projeto inteiro, incluindo controllers, configuração de segurança e infraestrutura que não são alvos de testes unitários. Ao isolar apenas a camada de negócio testada (use cases), a cobertura é de 100% em todos os indicadores.

ANÁLISE E CONCLUSÃO

Os resultados demonstram que a camada de aplicação do sistema EcoColeta está completamente coberta por testes unitários. Todos os cinco casos de teste foram executados

com sucesso, validando os seguintes comportamento críticos do sistema:

- Criação de solicitação de coleta com status inicial PENDENTE;
- Agendamento de coleta com transição de status para AGENDADO e publicação de evento;
- Atualização de status com publicação correta do evento no RabbitMQ;
- Tratamento adequado de exceções quando entidades não são encontradas.

A cobertura de 100% na camada de negócio, combinada com a documentação de contrato disponível via Swagger UI (SpringDoc OpenAPI 2.5.0), proporciona um nível de confiança adequado para o ciclo de desenvolvimento atual. A ausência de testes nas camadas de infraestrutura é uma escolha arquitetural intencional, seguindo o princípio de que componentes que dependem de recursos externos (banco de dados, broker de mensagens, contexto Spring) devem ser verificados por testes de integração ou por validação manual no ambiente de execução.